

AGENTS.md, pilot-r1 -> pilot-r2

Decompose the spec into focused GitHub issues before writing code. Work through them sequentially:

1. Create issues – one concern per issue, starting with project setup
- 2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
- +2. For each issue, follow this exact sequence:
 - + a. ``git checkout main && git pull && git checkout -b issue-N`` (one branch per issue, no exceptions)
 - + b. Write tests: ``git add tests/ && git commit -m "test: <description>"``
 - + c. Implement: ``git add src/ && git commit -m "feat: <description>"``
 - + d. Open PR: ``gh pr create --title "... " --body "Closes #N"``, merge, delete branch
3. Use conventional commits: ``test:`` for tests, ``feat:`` for implementation, ``docs:`` for documentation
- 4. Run ``tsc --noEmit`` before every PR – no type errors allowed
- +4. Before every PR, run these checks and fix all issues before opening:
 - + - ``tsc --noEmit`` – zero type errors
 - + - ``npx eslint src/ --max-warnings 0`` – zero lint warnings (includes complexity violations)

Package Quality

- Never combine multiple issues into one branch.

- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec – fix the code, not the test.
- + - Never modify a test to match your implementation.
- + - `\_tests/\_ref\_tests/` are immutable – they encode the acceptance criteria from the spec. Never touch them.
- + - If one of your own tests (outside `\_ref\_tests/`) has a bug, you may fix the test, but explain why in the commit message.
- + - Fix the code in `src/`, not the tests.
 - Never rewrite git history (no amend, squash, rebase, force-push).
 - Do not add dependencies beyond the dev toolchain (vitest, typescript).